MMDU MULLANA OFFICIAL SYLLABUS • BCSE-007 • UNIT 3

Unit 3: Queues & Linked Lists Topologies

Subject: Data Structures & Algorithms (BCSE-007) • **Exam Weightage:** 25 Marks • **Study Time:** 40 mins

🗺️ UNIT 3 SYLLABUS ROADMAP & TOPIC INDEX (TABLE OF CONTENTS)

3 Modules • 8 Topics • Pages 1–3

1. 1. Queue Topologies: Linear, Circular, Deque & Priority Pg. 1–2

- TOPIC 1. Linear Queue & The False Overflow Problem Pg. 1
- TOPIC 2. Circular Queue Solution (Modulo Arithmetic) Pg. 1
- TOPIC 3. Deque & Priority Queue Pg. 1

2. 2. Linked Lists Topologies: SLL, DLL, CLL & Header Lists Pg. 2

- TOPIC 1. Dynamic Linked Allocation vs Sequential Arrays Pg. 2
- TOPIC 2. Structural Comparison of Linked List Topologies Pg. 2

3. 3. Essential Pointer Rewiring Algorithms in C Pg. 2–3

- TOPIC 1. Singly Linked List: Insertion at Beginning Pg. 2
- TOPIC 2. Singly Linked List: Deleting a Node by Value Pg. 3
- TOPIC 3. Doubly Linked List: Insertion After Given Node Pg. 3

1. Queue Topologies: Linear, Circular, Deque & Priority

1. Linear Queue & The False Overflow Problem

A **Queue** enforces **FIFO (First-In, First-Out)** order. Additions occur at rear, deletions at front.
In an array-based linear queue, after repeated enqueues and dequeues, rear reaches MAX - 1 while free slots exist at the front. This is known as **False Overflow** or memory wastage.

2. Circular Queue Solution (Modulo Arithmetic)

A circular queue logically connects the last position back to index 0:

- **Empty Condition:** front == -1 && rear == -1
- **Full Condition:** (rear + 1) % MAX == front
- **Enqueue Operation:**

```
if ((rear + 1) % MAX == front) { printf("Queue Overflow
"); return; }
  if (front == -1) { front = 0; rear = 0; }
  else { rear = (rear + 1) % MAX; }
  queue[rear] = val;
```

- **Dequeue Operation:**

```
if (front == -1) { printf("Queue Underflow
"); return -1; }
  int val = queue[front];
  if (front == rear) { front = -1; rear = -1; } // Queue becomes empty
  else { front = (front + 1) % MAX; }
  return val;
```

3. Deque & Priority Queue

- **Deque (Double-Ended Queue):** Insertion and deletion can be performed at both the front and rear ends.

- *Input-Restricted Deque:* Insertion at one end only, deletion at both ends.
- *Output-Restricted Deque:* Deletion at one end only, insertion at both ends.

- **Priority Queue:** Elements are dequeued based on priority value rather than arrival order. Efficiently implemented using a **Binary Heap** (O(log n) insertion/extraction).

2. Linked Lists Topologies: SLL, DLL, CLL & Header Lists

1. Dynamic Linked Allocation vs Sequential Arrays

- **Sequential Array:** Fixed contiguous memory, O(1) random access by index, costly O(n) element shifts during insertions/deletions.

- **Linked List:** Non-contiguous dynamic nodes connected via pointers. Dynamic resizing, O(1) insertion/deletion given pointer reference, O(n) sequential access, additional memory overhead for storing pointer addresses.

2. Structural Comparison of Linked List Topologies

List Type	Node Structure	Traversal Direction	End Condition	Header Overhead		
Singly (SLL)	[Data	Next*]*	Unidirectional (Forward)	ptr->next == NULL	Single pointer (head)	
Doubly (DLL)	[Prev	Data	*Next]*	Bidirectional (Fwd/Bwd)	next == NULL, prev == NULL	Two pointers per node
Circular SLL	[Data	Next]	Circular Unidirectional	ptr->next == head	No NULL pointers	
Circular DLL	[*Prev*	Data	Next]	Circular Bidirectional	head->prev == last, last->next == head	Fast access to head & tail

| **Header List** | [Dummy Header] -> [Nodes] | As per underlying list | ptr->next == NULL | Dummy node storing list size/metadata |

3. Essential Pointer Rewiring Algorithms in C

1. Singly Linked List: Insertion at Beginning

```
struct Node {
    int data;
    struct Node* next;
};

void insertAtBeginning(struct Node** head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = *head; // Point new node to current head
    *head = newNode;       // Update head to new node
}
```

2. Singly Linked List: Deleting a Node by Value

```
void deleteNode(struct Node** head, int key) {
    struct Node *temp = *head, *prev = NULL;
    if (temp != NULL && temp->data == key) {
        *head = temp->next; // Head deleted
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return; // Key not found
    prev->next = temp->next;  // Unlink target node
    free(temp);               // Free heap memory
}
```

3. Doubly Linked List: Insertion After Given Node

```
void insertAfter(struct Node* prevNode, int val) {
    if (prevNode == NULL) return;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = prevNode->next;
    newNode->prev = prevNode;
    if (prevNode->next != NULL) {
        prevNode->next->prev = newNode;
    }
    prevNode->next = newNode;
}
```